

CSA0506 – Database Management
Systems CO5
ASSESSMENT TOOL 2 – Pseudocode Writing Task

Name: Dharanitharan G

Reg: 192511396

B.E CSE CS

1. [Apply] Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

```
ALGORITHM QueryProcessingPipeline(sqlQueryString)
BEGIN
    // Stage 1: Parsing
    tokens ← Lexer.tokenize(sqlQueryString)
    parseTree ← Parser.buildParseTree(tokens)
    IF parseTree is NULL THEN
        RAISE SyntaxError("Invalid SQL syntax")
    END IF

    // Stage 2: Semantic / Validation
    queryTree ← SemanticAnalyzer.validate(parseTree, SchemaCatalog)
    IF queryTree.hasUnknownTableOrColumn() THEN
        RAISE SemanticError("Unknown relation or attribute")
    END IF

    // Stage 3: Query Rewriting (view expansion, predicate simplification)
    logicalPlan ← Rewriter.expandViews(queryTree)
    logicalPlan ← Rewriter.pushDownSelections(logicalPlan)

    // Stage 4: Optimization
    candidatePlans ← Optimizer.generatePlans(logicalPlan)
    FOR EACH plan IN candidatePlans DO
        plan.cost ← CostEstimator.estimate(plan, Statistics)
    END FOR
    bestPlan ← SELECT plan FROM candidatePlans WITH MIN(plan.cost)

    // Stage 5: Execution Plan Generation
    executionPlan ← PlanGenerator.toPhysicalOperators(bestPlan)

    // Stage 6: Execution
    resultSet ← Executor.run(executionPlan)

    // Stage 7: Result Output
    RETURN FormatOutput(resultSet)
END
```

2. [Analyze] Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

```
ALGORITHM CostBasedJoinOrdering(relations R1, R2, ..., Rn)
BEGIN
    // bestPlan[S] holds the cheapest plan found so far for subset S of relations
    FOR EACH single relation Ri IN {R1,...,Rn} DO
        bestPlan[{Ri}] ← (accessPath = TableScan(Ri), cost = IOCost(Ri))
    END FOR

    FOR size = 2 TO n DO
        FOR EACH subset S OF {R1,...,Rn} WITH |S| = size DO
            bestPlan[S].cost ← INFINITY
            FOR EACH proper non-empty subset S1 OF S DO
                S2 ← S - S1
                IF bestPlan[S1] EXISTS AND bestPlan[S2] EXISTS THEN
                    FOR EACH joinMethod IN {NestedLoop, SortMerge, HashJoin} DO
                        candidateCost ← bestPlan[S1].cost + bestPlan[S2].cost
                                      + JoinCost(bestPlan[S1],
bestPlan[S2], joinMethod)
                        IF candidateCost < bestPlan[S].cost THEN
                            bestPlan[S] ← (left = bestPlan[S1], right =
bestPlan[S2],
                                      method = joinMethod, cost =
candidateCost)
                        END IF
                    END FOR
                END FOR
            END IF
        END FOR
    END FOR

    RETURN bestPlan[{R1,...,Rn}]
END

FUNCTION JoinCost(planA, planB, method)
BEGIN
    IF method = NestedLoop THEN
        RETURN planA.cardinality * CEIL(planB.pages / BufferPages) + planA.pages
    ELSE IF method = SortMerge THEN
        RETURN SortCost(planA) + SortCost(planB) + (planA.pages + planB.pages)
    ELSE IF method = HashJoin THEN
        RETURN 3 * (planA.pages + planB.pages)
    END IF
END
```

This follows the classic dynamic-programming approach used by System R style optimizers: the search space of join orders is built bottom-up over subsets of relations, each candidate join method's I/O cost is estimated using catalog statistics (page counts, cardinalities, buffer size), and only the cheapest plan for each subset is retained, which avoids re-evaluating dominated plans and keeps the search tractable even though the number of possible join orders grows factorially with the number of relations.

3. [Apply] Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

```
ALGORITHM BlockNestedLoopJoin(R, S, condition, B)
    // R = outer relation, S = inner relation, B = number of available buffer pages
BEGIN
    result ← EMPTY SET
    reserve 1 buffer page for S, 1 output buffer page, (B - 2) pages for blocks of R

    FOR EACH block Br OF (B - 2) pages of R (read sequentially) DO
        FOR EACH page Ps OF S DO
            FOR EACH tuple r IN Br DO
                FOR EACH tuple s IN Ps DO
                    IF condition(r, s) = TRUE THEN
                        result ← result ∪ {CONCAT(r, s)}
                        IF output buffer is full THEN
                            FLUSH output buffer TO disk
                        END IF
                    END IF
                END FOR
            END FOR
        END FOR
    END FOR
    RETURN result
END
```

Time Complexity Analysis

Let b_R and b_S be the number of disk pages occupied by R and S, and n_R and n_S be the number of tuples in R and S respectively, with B buffer pages available.

```
Number of outer blocks of R = CEIL( $b_R / (B - 2)$ )
Disk I/O cost =  $b_R + \text{CEIL}(b_R / (B - 2)) * b_S$ 
CPU comparison cost (worst case) =  $n_R * n_S$  comparisons
```

The disk I/O cost is what dominates in practice: R is read exactly once regardless of block size, but S is re-scanned once for every block of R that fits in the $(B - 2)$ reserved buffer pages, so cost falls as B grows. In the worst case, where only a single page of R is held in memory at a time ($B = 3$), the cost degrades to $b_R + b_R * b_S$ I/Os, whereas the naive tuple-at-a-time nested loop join would additionally incur $n_R * b_S$ I/Os, making the block-oriented version strictly more efficient for the same buffer allocation. The CPU cost of $n_R * n_S$ tuple comparisons is unaffected by buffering and represents the algorithm's inherent $O(n_R * n_S)$ time complexity.

4. [Analyze] Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

```
ALGORITHM DetectDeadlock(WFG)
    // WFG is a directed graph: node = transaction, edge  $T_i \rightarrow T_j$  means  $T_i$  waits for  $T_j$ 
BEGIN
    visited  $\leftarrow$  EMPTY SET
    inStack  $\leftarrow$  EMPTY SET

    FOR EACH transaction T IN WFG.nodes DO
        IF T NOT IN visited THEN
            IF DFS_HasCycle(T, WFG, visited, inStack) = TRUE THEN
                RETURN TRUE // deadlock detected
            END IF
        END IF
    END FOR
    RETURN FALSE // no deadlock
END

FUNCTION DFS_HasCycle(T, WFG, visited, inStack)
BEGIN
    visited  $\leftarrow$  visited  $\cup$  {T}
    inStack  $\leftarrow$  inStack  $\cup$  {T}

    FOR EACH neighbour N SUCH THAT ( $T \rightarrow N$ ) IN WFG.edges DO
        IF N IN inStack THEN
            RECORD cycle path (deadlock cycle found)
            RETURN TRUE
        ELSE IF N NOT IN visited THEN
            IF DFS_HasCycle(N, WFG, visited, inStack) = TRUE THEN
                RETURN TRUE
            END IF
        END IF
    END FOR

    inStack  $\leftarrow$  inStack - {T} // backtrack
    RETURN FALSE
END
```

The Wait-For Graph is rebuilt (or incrementally updated) whenever a transaction is blocked waiting for a lock held by another transaction. Running a depth-first search from every unvisited node and tracking the current recursion stack (inStack) allows a back-edge to a node already on the stack to be recognised as a cycle, which corresponds exactly to a set of transactions each waiting on the next in a closed loop, i.e. a deadlock. Once a cycle is found, the detector selects a victim transaction from the cycle (typically the youngest, or the one with the least work done) and aborts it to break the cycle and allow the remaining transactions to proceed.

5. [Apply] Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

```
ALGORITHM Basic2PL(Transaction T)
    // Each transaction moves through a Growing Phase (acquire only)
    // followed by a Shrinking Phase (release only), never both together.
BEGIN
    T.phase ← GROWING
    T.locksHeld ← EMPTY SET

    WHILE T has more operations DO
        op ← T.nextOperation()

        IF op requires READ(item X) THEN
            LockRequest(T, X, SHARED)
        ELSE IF op requires WRITE(item X) THEN
            LockRequest(T, X, EXCLUSIVE)
        ELSE IF op = COMMIT OR op = ABORT THEN
            T.phase ← SHRINKING
            ReleaseAllLocks(T)
            RETURN
        END IF
        Execute(op)
    END WHILE
END

FUNCTION LockRequest(T, X, mode)
BEGIN
    IF T.phase = SHRINKING THEN
        RAISE ProtocolViolation("Cannot acquire lock after shrinking phase started")
    END IF

    WHILE LockManager.isConflicting(X, mode, T) = TRUE DO
        ADD T TO WaitQueue(X)
        BLOCK(T) // wait phase
    END WHILE

    LockManager.grant(X, mode, T) // grant phase
    T.locksHeld ← T.locksHeld ∪ {(X, mode)}
END

FUNCTION ReleaseAllLocks(T)
BEGIN
    FOR EACH (X, mode) IN T.locksHeld DO
        LockManager.release(X, T)
        WAKE UP next compatible waiter(s) IN WaitQueue(X)
    END FOR
    T.locksHeld ← EMPTY SET
END
```

6. [Analyze] Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

```
ALGORITHM ThomasWriteRule(operation OP, item X, transaction T)
    // TS(T) = timestamp assigned to T at start
    // R-timestamp(X), W-timestamp(X) = timestamps of last read/write on X
BEGIN
    IF OP = READ(X) THEN
        IF TS(T) < W-timestamp(X) THEN
            ABORT(T) // T would read a value already overwritten "in the future"
            ROLLBACK(T)
            RESTART T WITH new timestamp
        ELSE
            Execute READ(X)
            R-timestamp(X) ← MAX(R-timestamp(X), TS(T))
        END IF

    ELSE IF OP = WRITE(X) THEN
        IF TS(T) < R-timestamp(X) THEN
            ABORT(T) // a younger transaction already read the value T would
overwrite
            ROLLBACK(T)
            RESTART T WITH new timestamp
        ELSE IF TS(T) < W-timestamp(X) THEN
            // Thomas' Write Rule: this write is obsolete, IGNORE it instead of
aborting
            SKIP the write (no-operation)
        ELSE
            Execute WRITE(X)
            W-timestamp(X) ← TS(T)
        END IF
    END IF
END
```

The distinguishing feature of Thomas' Write Rule, compared with the basic timestamp-ordering protocol, is the middle branch of the WRITE case: rather than aborting a transaction whose write arrives "too late" relative to W-timestamp(X), the obsolete write is simply discarded because no later read has depended on it, so ignoring it cannot violate serializability. This allows some schedules to be accepted as correct even though they are not strictly conflict-serializable, improving concurrency compared with strict timestamp ordering while still guaranteeing a serializable outcome.

7. [Apply] Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

```
ALGORITHM ARIES_Recovery(Log, LastCheckpoint)
BEGIN
    // ----- Phase 1: Analysis -----
    DirtyPageTable ← LastCheckpoint.DirtyPageTable
    TransactionTable ← LastCheckpoint.TransactionTable

    FOR EACH logRecord IN Log STARTING FROM LastCheckpoint DO
        IF logRecord.type IN {UPDATE} THEN
            IF logRecord.pageID NOT IN DirtyPageTable THEN
                DirtyPageTable[logRecord.pageID] ← logRecord.LSN
            END IF
        END IF
        IF logRecord.type IN {UPDATE, COMMIT, ABORT} THEN
            TransactionTable[logRecord.transID].lastLSN ← logRecord.LSN
            IF logRecord.type = COMMIT THEN
                TransactionTable[logRecord.transID].status ← COMMITTED
            ELSE IF logRecord.type = ABORT THEN
                TransactionTable[logRecord.transID].status ← ABORTING
            END IF
        END IF
    END FOR
    Lossers ← { T IN TransactionTable | T.status ≠ COMMITTED }

    // ----- Phase 2: Redo -----
    redoLSN ← MIN(LSN IN DirtyPageTable.values())
    FOR EACH logRecord IN Log STARTING FROM redoLSN DO
        IF logRecord.type = UPDATE
            AND logRecord.pageID IN DirtyPageTable
            AND logRecord.LSN ≥ DirtyPageTable[logRecord.pageID] THEN
            IF PageLSN(logRecord.pageID) < logRecord.LSN THEN
                REAPPLY logRecord.change TO logRecord.pageID
                PageLSN(logRecord.pageID) ← logRecord.LSN
            END IF
        END IF
    END FOR

    // ----- Phase 3: Undo -----
    ToUndo ← { T.lastLSN | T IN Lossers }
    WHILE ToUndo IS NOT EMPTY DO
        LSN ← MAX(ToUndo); REMOVE LSN FROM ToUndo
        logRecord ← Log.get(LSN)
        IF logRecord.type = UPDATE THEN
            UNDO logRecord.change // apply the before-image
            WRITE Compensation Log Record (CLR) with undoNextLSN = logRecord.prevLSN
            IF logRecord.prevLSN EXISTS THEN
                ADD logRecord.prevLSN TO ToUndo
            END IF
        END IF
    END WHILE
    RETURN "Recovery Complete"
END
```

8. [Apply] Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

```
ALGORITHM ShadowPagingTransaction(Transaction T)
BEGIN
    // At transaction start, the current page table is a copy of the shadow page table
    CurrentPageTable ← COPY(ShadowPageTable)

    WHILE T has more operations DO
        op ← T.nextOperation()
        IF op = WRITE(pageID, newData) THEN
            newPage ← ALLOCATE new disk block
            WRITE newData TO newPage
            CurrentPageTable[pageID] ← newPage.address // redirect, old page
untouched
        ELSE IF op = READ(pageID) THEN
            RETURN READ FROM CurrentPageTable[pageID]
        ELSE IF op = COMMIT THEN
            COMMIT_ShadowPaging(T)
            RETURN
        ELSE IF op = ABORT THEN
            ROLLBACK_ShadowPaging(T)
            RETURN
        END IF
    END WHILE
END

FUNCTION COMMIT_ShadowPaging(T)
BEGIN
    FLUSH all modified pages referenced by CurrentPageTable TO disk
    FLUSH CurrentPageTable itself TO a new disk block
    // Atomic switch: overwrite the single pointer to the page table
    ATOMICALLY SET DiskHeader.pageTablePointer ← address of new CurrentPageTable
    ShadowPageTable ← CurrentPageTable // new shadow becomes the committed state
    RECLAIM old shadow pages that are no longer referenced
END

FUNCTION ROLLBACK_ShadowPaging(T)
BEGIN
    // Simply discard the current page table; nothing was ever made visible
    DEALLOCATE all pages allocated by T that are only referenced by CurrentPageTable
    CurrentPageTable ← DISCARD
    // DiskHeader.pageTablePointer still points at the untouched ShadowPageTable
END

FUNCTION RecoverAfterCrash()
BEGIN
    // On restart, DiskHeader.pageTablePointer still points to the last committed
    // shadow page table, since the switch is a single atomic disk write.
    ShadowPageTable ← READ(DiskHeader.pageTablePointer)
    // Any pages written by an in-flight, uncommitted transaction are simply
    // unreferenced and require no explicit undo.
END
```

Because the current page table is only made visible by a single atomic write of the disk header pointer at commit time, a crash at any point before that write leaves the on-disk shadow page table exactly as it was before the

transaction began, so recovery requires no REDO or UNDO log processing at all — the old shadow copy is simply used as-is. The main costs are the extra disk space and I/O needed to copy the page table on every transaction and the difficulty of supporting concurrent transactions, which is why shadow paging has largely been superseded by log-based schemes such as ARIES in practice.

9. [Apply] Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

```
ALGORITHM ImmediateUpdateRecovery(Log)
    // Immediate Update: writes may be flushed to disk before commit,
    // so both UNDO (for uncommitted changes) and REDO (for committed
    // changes not yet reflected on disk) may be required.
BEGIN
    UndoList ← EMPTY SET // transactions active at crash time
    RedoList ← EMPTY SET // transactions committed before crash

    // ----- Pass 1: scan log forward -----
    FOR EACH logRecord IN Log (in order) DO
        IF logRecord.type = START THEN
            ADD logRecord.transID TO UndoList
        ELSE IF logRecord.type = COMMIT THEN
            REMOVE logRecord.transID FROM UndoList
            ADD logRecord.transID TO RedoList
        END IF
    END FOR

    // ----- Pass 2: REDO (forward, oldest to newest) --
    FOR EACH logRecord IN Log (in order) DO
        IF logRecord.type = UPDATE AND logRecord.transID IN RedoList THEN
            WRITE logRecord.afterImage TO logRecord.dataItem // reapply the change
        END IF
    END FOR

    // ----- Pass 3: UNDO (backward, newest to oldest) -
    FOR EACH logRecord IN Log (in reverse order) DO
        IF logRecord.type = UPDATE AND logRecord.transID IN UndoList THEN
            WRITE logRecord.beforeImage TO logRecord.dataItem // reverse the change
        END IF
        IF logRecord.type = START AND logRecord.transID IN UndoList THEN
            APPEND ABORT log record FOR logRecord.transID
        END IF
    END FOR
    RETURN "Recovery Complete"
END
```

10. [Analyze] Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

```
CLASS ConnectionPoolManager
    availablePool ← EMPTY QUEUE
    inUsePool ← EMPTY SET
    maxPoolSize ← N

    FUNCTION Initialize()
    BEGIN
        FOR i = 1 TO maxPoolSize DO
            conn ← CreatePhysicalConnection(dbURL, credentials)
            ENQUEUE conn INTO availablePool
        END FOR
    END

    FUNCTION GetConnection()
    BEGIN
        WHILE availablePool IS EMPTY DO
            IF TotalConnections() < maxPoolSize THEN
                RETURN CreatePhysicalConnection(dbURL, credentials)
            ELSE
                WAIT until a connection is released (with timeout)
            END IF
        END WHILE
        conn ← DEQUEUE FROM availablePool
        ADD conn TO inUsePool
        RETURN conn
    END

    FUNCTION ExecuteQuery(conn, sqlStatement, params)
    BEGIN
        TRY
            conn.beginTransaction()
            result ← conn.execute(sqlStatement, params)
            conn.commit()
            RETURN result
        CATCH SQLException AS ex
            conn.rollback()
            LOG("Query failed, transaction rolled back: " + ex.message)
            RAISE ex
        FINALLY
            ReleaseConnection(conn)
        END TRY
    END

    FUNCTION ReleaseConnection(conn)
    BEGIN
        IF conn.isValid() = TRUE THEN
            REMOVE conn FROM inUsePool
            ENQUEUE conn INTO availablePool
            NOTIFY any thread waiting in GetConnection()
        ELSE
            REMOVE conn FROM inUsePool
            CLOSE conn
        END IF
    END
END
```

```
        ENQUEUE CreatePhysicalConnection(dbURL, credentials) INTO availablePool
    END IF
END
END CLASS
```